

Cloud Engineering Guide

1. Core Cloud Concepts: IaaS, PaaS, and SaaS

Infrastructure as a Service (IaaS) gives you raw compute, storage, and networking primitives — virtual machines, block storage, virtual networks — and you are responsible for the operating system, runtime, and everything above it. Amazon EC2, Google Compute Engine, and Azure Virtual Machines are classic IaaS offerings. Platform as a Service (PaaS) abstracts away the operating system and runtime management, letting you deploy application code directly (examples include AWS Elastic Beanstalk, Google App Engine, and Heroku).

Software as a Service (SaaS) delivers a complete, ready-to-use application over the internet with no infrastructure management at all from the consumer (examples: Gmail, Salesforce). A cloud engineer typically works most heavily with IaaS and PaaS layers, plus a growing category of managed services (managed databases, managed Kubernetes, managed message queues) that sit between IaaS and PaaS in terms of how much operational responsibility they remove.

Understanding the shared responsibility model is essential: cloud providers secure the underlying physical infrastructure, but customers remain responsible for securing their own configuration, identity and access management, data, and application code. Misconfigured storage buckets and overly permissive IAM roles — not provider infrastructure failures — are the leading cause of cloud security incidents.

Cloud Engineering Guide

2. Core Services Across AWS, GCP, and Azure

Compute: AWS EC2 / Lambda, GCP Compute Engine / Cloud Functions, and Azure Virtual Machines / Functions cover both long-running servers and event-driven serverless functions. Serverless functions are billed per invocation and scale to zero, making them well suited to unpredictable or low-traffic workloads, while traditional VMs remain a better fit for steady, predictable, long-running workloads.

Storage: object storage (S3, Cloud Storage, Blob Storage) is the default choice for static assets, backups, and large unstructured files, priced per gigabyte stored and per request; block storage (EBS, Persistent Disk) attaches to a single VM like a traditional hard drive and is used for database volumes and other performance-sensitive workloads.

Managed databases (RDS/Aurora, Cloud SQL, Azure Database) handle automated backups, patching, and failover for relational databases, removing most day-to-day database administration burden compared to self-managing a database on a raw VM. Networking primitives (VPCs, subnets, security groups/firewall rules, load balancers) are conceptually similar across all three major providers even though the exact terminology differs.

Cloud Engineering Guide

3. Infrastructure as Code

Infrastructure as Code (IaC) means defining your cloud resources in version-controlled configuration files rather than clicking through a web console, so that infrastructure changes go through the same review, testing, and rollback discipline as application code. Terraform (cloud-agnostic, HCL syntax) and AWS CloudFormation (AWS-specific, YAML/JSON) are the two most common tools; both work by comparing a declared desired state against the actual state of your cloud account and computing the changes needed to reconcile them.

A core IaC skill is understanding 'plan before apply' — reviewing exactly what will change, be created, or be destroyed before executing it, since a wrong apply can delete a production database or take down a load balancer. State management (Terraform's state file, tracking what resources it currently owns) must be stored securely and shared safely across a team, typically in a remote backend like an S3 bucket with locking, rather than a local file.

Modularizing IaC into reusable components (a 'VPC module', a 'database module') mirrors good software engineering practice and lets a small team manage a growing number of environments (dev, staging, prod) without duplicating hundreds of lines of configuration for each one.

Cloud Engineering Guide

4. Containers and Orchestration

Docker packages an application together with its dependencies and runtime into a single portable image, guaranteeing that the same artifact runs identically in a developer's laptop, a CI pipeline, and production — eliminating the classic 'works on my machine' failure mode. A well-built Docker image uses a minimal base image, a multi-stage build to keep the final image small, and never bakes secrets into image layers.

Kubernetes orchestrates many containers across many machines: it schedules containers onto available nodes, restarts ones that crash, load-balances traffic across replicas, and can automatically scale the number of running replicas based on CPU or custom metrics. Core Kubernetes concepts every cloud engineer should know include Pods (the smallest deployable unit), Deployments (managing replica sets and rolling updates), Services (stable networking for a set of pods), and ConfigMaps/Secrets (externalized configuration).

Managed Kubernetes offerings (EKS, GKE, AKS) remove the operational burden of running the Kubernetes control plane itself, letting a team focus on workloads rather than cluster maintenance — this is usually the right default choice over self-managing Kubernetes unless there is a specific, well-understood reason not to.

Cloud Engineering Guide

5. Cloud Security and Cost Optimization

The principle of least privilege should govern every IAM role and policy: grant only the specific permissions a service or person needs, scoped to the specific resources they need them on, and avoid wildcard (*) permissions except in tightly justified cases. Enable audit logging (AWS CloudTrail, GCP Cloud Audit Logs) on every account so that any configuration change or access event can be traced after the fact.

Encrypt data at rest (enabled by default on most managed storage and database services today) and in transit (TLS everywhere, including internal service-to-service traffic where feasible). Rotate credentials and access keys regularly, and prefer short-lived, role-based credentials (assumed roles, workload identity) over long-lived static access keys wherever the platform supports it.

Cost optimization starts with visibility: tag every resource with an owner and environment, and review a cloud cost dashboard regularly rather than only when a bill spikes. Right-size compute instances based on actual observed utilization, use auto-scaling so you are not paying for peak capacity around the clock, and use reserved or spot/preemptible instances for predictable or fault-tolerant workloads respectively to cut compute costs significantly.

Cloud Engineering Guide

6. Observability and Incident Response

Observability rests on three pillars: metrics (numeric time series like CPU usage, request rate, and error rate), logs (structured, searchable records of discrete events), and traces (following a single request as it moves across multiple services). A cloud engineer should be comfortable setting up dashboards and alerts on all three, not just watching a single 'is it up' health check.

Alerts should be actionable and tied to user-facing symptoms (elevated error rate, rising latency, failed health checks) rather than firing on every minor internal metric fluctuation, since alert fatigue causes real incidents to be missed among noise. Define clear on-call runbooks describing the first diagnostic steps for common alert types, so incident response does not depend on one specific engineer's memory.

After any significant incident, write a blameless postmortem that documents the timeline, root cause, and concrete follow-up action items — the goal is a system and process improvement, not attributing fault to an individual. Teams that consistently practice postmortems build measurably more reliable systems over time than teams that treat each incident as a one-off.